Python Programming - Beginner Level Notes

Prepared for: Bikkad IT Institute Students
Topic: Errors & Exception Handling
Date: February 2026

---

1. Errors

What are Errors?

Errors are problems in code that prevent the program from running correctly.

Types of Errors:

- Syntax Errors
- Runtime Errors (Exceptions)
- Logical Errors

---

2. Syntax Errors

What is Syntax Error?

Errors in the structure of code. Python cannot understand the code.

Common Causes:

- Missing colon (:)
- Missing parentheses
- Wrong indentation
- Misspelled keywords

Code Examples

Example 1: Missing Colon

Error - missing colon

```
if 5 > 3
print("Five is greater")
```

SyntaxError: invalid syntax

Correct:
if 5 > 3:
print("Five is greater")

Example 2: Wrong Indentation

Error - wrong indentation

def greet():
print("Hello")

IndentationError: expected an indented block

Correct:
def greet():
print("Hello")

Example 3: Unclosed Parentheses

Error - unclosed parentheses

print("Hello World"

SyntaxError: unexpected EOF while parsing

Correct:
print("Hello World")

---

3. Index Error

What is Index Error?

Occurs when trying to access an index that doesn't exist in a sequence.

Code Examples

Example 1: List Index Error
numbers = [10, 20, 30, 40]

Error - index 5 doesn't exist

print(numbers[5])

IndexError: list index out of range

Solution:
numbers = [10, 20, 30, 40]

Check index before accessing

if 5 < len(numbers):
print(numbers[5])
else:
print("Index out of range")

Example 2: String Index Error
text = "Python"

Error - index 10 doesn't exist

print(text[10])

IndexError: string index out of range

Example 3: Negative Index Error
numbers = [10, 20, 30]

Error - negative index too large

print(numbers[-5])

IndexError: list index out of range

---

4. Module Not Found Error

What is Module Not Found Error?

Occurs when trying to import a module that is not installed.

Code Examples

Example 1: Module Not Installed

Error - pandas not installed

import pandas

ModuleNotFoundError: No module named 'pandas'

Solution:

Install module first: pip install pandas

import pandas

Example 2: Wrong Module Name

Error - wrong spelling

import numpyy

ModuleNotFoundError: No module named 'numpyy'

Correct:
import numpy

---

5. Key Error

What is Key Error?

Occurs when trying to access a dictionary key that doesn't exist.

Code Examples

Example 1: Missing Key
student = {"name": "Rahul", "age": 25}

Error - 'marks' key doesn't exist

print(student["marks"])

KeyError: 'marks'

Solution 1: Use get() method
student = {"name": "Rahul", "age": 25}

No error - returns None

print(student.get("marks")) # None

With default value

print(student.get("marks", 0)) # 0

Solution 2: Check before accessing
student = {"name": "Rahul", "age": 25}

if "marks" in student:
print(student["marks"])
else:
print("Key not found")

---

6. Type Error & Value Error

Type Error

Occurs when an operation is performed on wrong data type.

Code Examples - Type Error

Example 1: Adding String and Integer

Error - cannot add string and int

result = "5" + 10

TypeError: can only concatenate str (not "int") to str

Solution:
result = int("5") + 10 # 15

OR

result = "5" + str(10) # "510"

Example 2: Wrong Argument Type

Error - len() expects sequence

length = len(123)

TypeError: object of type 'int' has no len()

Correct:
length = len("123") # 3

Value Error

Occurs when function receives correct type but inappropriate value.

Code Examples - Value Error

Example 1: Converting Invalid String

Error - cannot convert "abc" to int

number = int("abc")

ValueError: invalid literal for int() with base 10: 'abc'

Solution:
try:
number = int("abc")
except ValueError:
print("Cannot convert to integer")

Example 2: Unpacking Error

Error - too many values to unpack

a, b = [1, 2, 3]

ValueError: too many values to unpack (expected 2)

Correct:
a, b, c = [1, 2, 3]

---

7. Name Error

What is Name Error?

Occurs when using a variable that has not been defined.

Code Examples

Example 1: Variable Not Defined

Error - 'x' not defined

print(x)

NameError: name 'x' is not defined

Solution:
x = 10
print(x) # 10

Example 2: Misspelled Variable
name = "Rahul"

Error - wrong spelling

print(namee)

NameError: name 'namee' is not defined

Example 3: Function Not Defined

Error - function not defined

result = multiply(5, 3)

NameError: name 'multiply' is not defined

Solution:
def multiply(a, b):
return a * b

result = multiply(5, 3)

---

8. Attribute Error

What is Attribute Error?

Occurs when trying to access an attribute or method that doesn't exist.

Code Examples

Example 1: Method Doesn't Exist
numbers = [1, 2, 3]

Error - no 'add' method for list

numbers.add(4)

AttributeError: 'list' object has no attribute 'add'

Correct:
numbers = [1, 2, 3]
numbers.append(4) # Correct method

Example 2: Attribute Doesn't Exist
text = "Python"

Error - strings don't have 'length' attribute

print(text.length)

AttributeError: 'str' object has no attribute 'length'

Correct:
print(len(text)) # 6

---

9. Exception

What is an Exception?

An error that occurs during program execution (runtime error).

Difference Between Error and Exception:

Error	Exception
Syntax related	Runtime related
Cannot be handled	Can be handled
Program won't run	Program runs until exception

Table 1: Error vs Exception

---

10. Why We Need to Handle Exception

Reasons:

- Prevent program crash
- Provide user-friendly error messages
- Graceful error recovery
- Log errors for debugging
- Continue program execution

Code Example

Without Exception Handling:
num = int(input("Enter number: ")) # User enters "abc"

Program crashes with ValueError

print("This line never executes")

With Exception Handling:
try:
num = int(input("Enter number: "))
print(f"Number: {num}")
except ValueError:
print("Please enter valid number")

```
print("Program continues...") # This executes
```

---

11. How to Handle Exception - Try & Except Block

Syntax

```
try:
# Code that might cause error
pass
except:
# Code to handle error
pass
```

Code Examples

Example 1: Basic Try-Except

```
try:
result = 10 / 0
except:
print("Error occurred")
```

Example 2: Division by Zero

```
try:
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
result = num1 / num2
print(f"Result: {result}")
except:
print("Error: Cannot divide")
```

Example 3: File Handling

```
try:
file = open("data.txt", "r")
content = file.read()
print(content)
except:
print("File not found")
```

---

12. Handling Multiple Errors

Using Multiple Except Blocks

Code Examples

Example 1: Multiple Except Blocks

```
try:
numbers = [10, 20, 30]
index = int(input("Enter index: "))
print(numbers[index])
except IndexError:
print("Error: Index out of range")
```

```
except ValueError:
print("Error: Enter valid integer")
except:
print("Error: Something went wrong")
```

Example 2: Division with Multiple Errors

```
try:
num1 = int(input("Enter number 1: "))
num2 = int(input("Enter number 2: "))
result = num1 / num2
print(f"Result: {result}")
except ValueError:
print("Error: Enter valid numbers")
except ZeroDivisionError:
print("Error: Cannot divide by zero")
except:
print("Error: Unknown error occurred")
```

---

13. Handling Specific Error

Catching Error Object

Code Examples

Example 1: Print Error Message

```
try:
num = int("abc")
except ValueError as e:
print(f"Error occurred: {e}")
# Output: Error occurred: invalid literal for int()...
```

Example 2: Multiple Errors in One Block

```
try:
file = open("data.txt")
content = file.read()
number = int(content)
except (FileNotFoundError, ValueError) as e:
print(f"Error: {e}")
```

Example 3: Detailed Error Handling

```
try:
student = {"name": "Rahul"}
print(student["marks"])
except KeyError as e:
print(f"Key {e} not found in dictionary")
```

---

14. Try - Except - Else

Else Block

Executes only if NO exception occurs in try block.

Code Examples

Example 1: Division with Else

```
try:
num1 = int(input("Enter number 1: "))
num2 = int(input("Enter number 2: "))
result = num1 / num2
except ValueError:
print("Enter valid numbers")
except ZeroDivisionError:
print("Cannot divide by zero")
else:
print(f"Division successful: {result}")
```

Example 2: File Reading with Else

```
try:
file = open("data.txt", "r")
except FileNotFoundError:
print("File not found")
else:
content = file.read()
print(f"File content: {content}")
file.close()
```

---

15. Try - Except - Else - Finally

Finally Block

Always executes, whether exception occurs or not.

Use Cases:

- Close files
- Release resources
- Cleanup operations
- Database connections

Code Examples

Example 1: File Handling with Finally

```
try:
file = open("data.txt", "r")
content = file.read()
print(content)
except FileNotFoundError:
```

```
print("File not found")
else:
print("File read successfully")
finally:
print("Closing file...")
try:
file.close()
except:
pass
```

Example 2: Database Connection

```
try:
# Connect to database
print("Connecting to database...")
result = 10 / 2
print(f"Result: {result}")
except ZeroDivisionError:
print("Division error")
else:
print("Operation successful")
finally:
print("Closing database connection...")
```

Example 3: Complete Flow

```
def divide_numbers():
try:
num1 = int(input("Enter number 1: "))
num2 = int(input("Enter number 2: "))
result = num1 / num2
except ValueError:
print("Error: Enter valid integers")
except ZeroDivisionError:
print("Error: Cannot divide by zero")
else:
print(f"Result: {result}")
return result
finally:
print("Function execution completed")

divide_numbers()
```

---

16. Raise Exception

What is Raise?

Manually trigger an exception in your code.

Code Examples

Example 1: Raise ValueError

```
def check_age(age):
if age < 0:
raise ValueError("Age cannot be negative")
elif age < 18:
raise ValueError("Age must be 18 or above")
else:
print(f"Age {age} is valid")

try:
check_age(-5)
except ValueError as e:
print(f"Error: {e}")
```

Example 2: Raise with Custom Message

```
def withdraw_money(balance, amount):
if amount > balance:
raise ValueError("Insufficient balance")
return balance - amount

try:
remaining = withdraw_money(1000, 1500)
except ValueError as e:
print(f"Error: {e}")
```

Example 3: Re-raise Exception

```
try:
num = int("abc")
except ValueError:
print("Caught ValueError")
raise # Re-raise the same exception
```

---

17. Creating Custom Exception

How to Create Custom Exception

Create a class that inherits from `Exception` class.

Code Examples

Example 1: Basic Custom Exception

```
class InvalidAgeError(Exception):
pass

def check_voting_age(age):
if age < 18:
raise InvalidAgeError("You must be 18 or older to vote")
else:
print("You are eligible to vote")
```

```
try:
check_voting_age(15)
except InvalidAgeError as e:
print(f"Error: {e}")
```

Example 2: Custom Exception with Constructor

```
class InsufficientBalanceError(Exception):
def init(self, balance, amount):
self.balance = balance
self.amount = amount
self.message = f"Insufficient balance. Balance: {balance}, Required: {amount}"
super().init(self.message)

def withdraw(balance, amount):
if amount > balance:
raise InsufficientBalanceError(balance, amount)
return balance - amount

try:
remaining = withdraw(1000, 1500)
except InsufficientBalanceError as e:
print(e.message)
```

Example 3: Multiple Custom Exceptions

```
class NegativeValueError(Exception):
"""Raised when negative value is entered"""
pass

class ZeroValueError(Exception):
"""Raised when zero is entered"""
pass

def calculate_percentage(obtained, total):
if obtained < 0 or total < 0:
raise NegativeValueError("Marks cannot be negative")
if total == 0:
raise ZeroValueError("Total marks cannot be zero")
return (obtained / total) * 100

try:
percentage = calculate_percentage(85, 0)
except NegativeValueError as e:
print(f"Error: {e}")
except ZeroValueError as e:
print(f"Error: {e}")
else:
print(f"Percentage: {percentage}%")
```

Practice Exercises

Exercise 1: Handle User Input

Write a program to take integer input and handle ValueError.

Exercise 2: List Index

Create a program that handles IndexError when accessing list elements.

Exercise 3: Dictionary Keys

Write a program to safely access dictionary keys using try-except.

Exercise 4: File Operations

Create a program that handles FileNotFoundError and reads a file.

Exercise 5: Division Calculator

Build a calculator that handles ZeroDivisionError and ValueError.

Exercise 6: Custom Exception

Create a custom exception for validating student marks (0-100).

Exercise 7: Finally Block

Write a program that uses try-except-else-finally to manage resources.

Exercise 8: Multiple Exceptions

Handle at least 3 different exceptions in a single program.

Quick Reference

Exception Type	When It Occurs
SyntaxError	Wrong code structure
IndexError	Invalid index access
KeyError	Invalid dictionary key
ValueError	Wrong value type
TypeError	Wrong data type
NameError	Undefined variable
AttributeError	Invalid attribute
ZeroDivisionError	Division by zero

Table 2: Common Python Exceptions

Block	Purpose
try	Code that might raise exception
except	Handle the exception
else	Execute if no exception
finally	Always execute
raise	Manually raise exception

Table 3: Exception Handling Blocks

Key Concepts Summary

Exception Handling Benefits:

- Prevents program crashes
- Better user experience
- Error logging and debugging
- Resource cleanup (finally block)
- Graceful error recovery

Best Practices:

- Handle specific exceptions, not generic
- Use finally for cleanup operations
- Don't suppress exceptions silently
- Provide meaningful error messages
- Create custom exceptions for specific scenarios

Try-Except-Else-Finally Flow:

- Try: Attempt risky code
- Except: Handle errors if they occur
- Else: Execute if no errors (optional)
- Finally: Always execute (cleanup)

End of Notes

Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: February 2026